

1 DesignGym — CipherSchools LLD Practice Lab

DesignGym is a focused practice loop for CipherSchools learners building software-development and interview-ready design judgment:

**Choose a problem → make the reasoning visible → submit → receive explainable feedback
→ review weaknesses → retry.**

The product is intentionally narrow: it is a practice-and-feedback layer that complements CipherSchools lessons instead of pretending to be a complete LMS.

1.1 Live prototype

Open the verified prototype here: [DesignGym — CipherSchools LLD Practice Lab](#).

The link opens the current WebDev preview. For GitHub publication, push this repository to your own account after cloning or downloading it; the complete source, documentation, tests, and PDF submission bundle are already included.

1.2 Product loop

The learner chooses Parking Lot, Vending Machine, Elevator System, or Library Management; writes assumptions, classes, responsibilities, interactions, edge cases, and trade-offs; submits; receives evidence-linked rubric feedback; sees recurring weaknesses; and retries without overwriting history.

Attempt status is explicit and guarded:

```
Draft → Submitted → Evaluating → Completed
                        ↘ Failed
```

Illegal transitions such as Draft → Completed throw `InvalidStateTransitionError`.

1.3 Run locally

```
pnpm install
pnpm dev
```

For quality checks:

```
pnpm check
pnpm build
pnpm test
```

The full-stack WebDev scaffold provides React, Express, tRPC, Drizzle, SQL persistence, Manus OAuth, secure sessions, and credential authentication. The core domain tests remain runnable independently through Vitest.

1.4 Reviewer walkthrough

1. Open the overview and choose **Parking Lot**.
2. Continue with the seeded demo or start a blank rep.
3. Edit the six evidence sections while keeping the problem requirements visible.

4. Submit and inspect the seven-criterion review with evidence, concerns, suggestions, and next focus.
5. Open **Attempt history** to see preserved attempts and recurring weak criteria.
6. Use the sidebar account menu to sign in or create an account, then open account settings to choose a CipherSchools learning track, change a password, request recovery, and verify a contact.

1.5 Architecture

The domain lives in `client/src/lib/designgym.ts`. `Attempt` owns a guarded lifecycle; `Submission` is the submission-format boundary; `EvaluatorStrategy` is the judgment boundary; and `EvaluationEngine` coordinates asynchronous evaluation, fallback, and terminal state changes.

`StructuredWrittenSubmission` is the MVP. `DiagramSubmission` demonstrates the text-to-class-diagram change test without requiring changes to `Attempt` or `EvaluationEngine`. A `HumanReviewEvaluator`, `RuleBasedEvaluator`, or LLM adapter can implement `EvaluatorStrategy` without changing practice logic.

The evaluator contract is explainable and structured:

```
Criterion → Score → Evidence → Concern → Suggestion
```

`parseRubricEvaluation` validates structured JSON output. If a primary evaluator times out or fails, `EvaluationEngine` runs `DeterministicRubricEvaluator`, records the fallback source and failure details, and completes safely. If both evaluators fail, the attempt reaches `Failed` without losing the submission.

1.6 Backend and persistence

The full-stack app includes SQL-backed `users`, `attempts`, and `authTokens` tables; credential login with username, email, or phone; bcrypt password hashing; signed `httpOnly` sessions; Manus OAuth; protected `tRPC` procedures; profile and learning-track updates; password change/reset; and email/phone verification token flows.

For production deployment, connect an email provider and SMS provider to deliver verification and reset tokens rather than displaying the demo token in the UI.

1.7 Assignment deliverables

- [docs/RESEARCH.md](#) — research note and CipherSchools product decision.
- [docs/DESIGN.md](#) — domain model, guarded state machine, failure handling, and Section 9 change tests.
- [AI_USAGE.md](#) — four explicit AI-assisted engineering decisions.
- `client/src/lib/designgym.ts` — domain types, state machine, submission strategies, evaluator strategies, fallback engine, seed data, and aggregation.
- `client/src/lib/designgym.test.ts` — state, payload, strategy, fallback, JSON parsing, and improvement-loop tests.
- `server/account.auth.test.ts` and `server/auth.logout.test.ts` — protected account and session tests.

1.8 Integrity status

The project is synchronized around the same architecture described in the assignment documents: a monolithic, testable attempt state machine; explicit strategy boundaries; deterministic and judgment-

oriented feedback separation; structured JSON validation; and graceful evaluator failure handling. `pnpm check`, `pnpm build`, and `pnpm test` are the required final gates.